

函数基础：声明、调用、返回值

先声明再调用——Python 从上往下扫描函数定义，声明必须在调用前面。日常写法：前面全部声明函数，末尾统一调用。

return 做两件事：把结果送出去，同时终止函数。函数里遇到 return，后面的代码全不执行。没写 return 或只写了空 return，返回 None。

多个返回值：一个 return 后面跟多个值，Python 自动打包成元组。用对应数量的变量接收，就拆开成各自类型；用 1 个变量接收，就是元组。

匿名函数 lambda：没有名字，冒号前是形参，冒号后是返回值，自带 return。只能写简单逻辑，复杂判断和循环还是得用 def。

pass 占位符：先搭结构再填逻辑，保证程序能跑。

```
In [4]: #定义 一个函数来计算两个数的和
def sum(a, b):
    sum = a + b #计算 a 和 b 的和
    return sum #返回 a 和 b 的和
def main():
    print(sum(3, 5))#调用 sum 函数并打印结果
if __name__ == "__main__":
    main()
```

8

```
In [5]: #匿名函数
add = lambda x, y: x + y #定义一个匿名函数来计算两个数的和
print(add(3, 5)) #调用匿名函数并打印结果
```

8

```
In [10]: def print_info(name, age, sex="男"): # 在定义函数时，给 sex 赋上默认值 "男"
    print(f"姓名: {name}, 年龄: {age}, 性别: {sex}")
    print_info("张三", 25) # 不传 sex, 自动使用默认值 "男"
    print_info("李四", 30, "女") # 传入 "女", 覆盖默认值
```

姓名: 张三, 年龄: 25, 性别: 男
姓名: 李四, 年龄: 30, 性别: 女

参数类型	声明方式	调用规则
必须参数	def f(a, b)	顺序传，缺了报 TypeError
关键字参数	调用时 f(age=16)	一旦出现，后面全得用关键字
默认参数	def f(a, h=1.88)	传了用传入值，没传用默认值
不定长参数	*args / **kwargs	一颗星打包成元组，两颗星打包成字典

打包 vs 解包：声明时用星号叫打包（多个实参 → 元组/字典），调用时用星号叫解包（元组/字典 → 多个实参）。元组一颗星，字典两颗星。位置陷阱：不定长参数放前面会"吃掉"所有实参，后面的必须参数只能用关键字参数传。

```
In [23]: # 假设这是你定义的函数
def total_price(cut_off, *prices):
```

```

    total = sum(prices)
    if total > cut_off:
        return total - cut_off
    else:
        return total
# 准备数据
prices_list = [10, 20, 30, 15, 25] # 总价为 100
cut_off = 65 # 假设满减门槛是 65

# ✅ 正确的调用方式:
# 1. cut_off 使用关键字参数传值
# 2. 列表前加 * 进行解包, 且不能加等号
ret = total_price(cut_off, *prices_list) # ✅ 正确的调用方式

print(ret) # 输出 35 (因为总价 100 > 65, 所以返回 100 - 65 = 35)

```

35

```

In [28]: def func1(num):
        if num == 1:
            print("1")
            return 99
        elif num == 2:
            print("2")
            return 88
        func1(1) # 输出 "1", 返回 99
        #func1(2) # 输出 "2", 返回 88

```

1

Out[28]: 99

```

In [34]: def get_season(month):
        if month in [3, 4, 5]:
            return "春季"
        elif month in [6, 7, 8]:
            return "夏季"
        elif month in [9, 10, 11]:
            return "秋季"
        elif month in [12, 1, 2]:
            return "冬季"
        else:
            return "无效的月份"
        month = int(input("请输入月份(1-12):")) # 获取用户输入的月份
        get_season(month) # 调用函数并输出结果

```

Out[34]: '无效的月份'

```

In [36]: def fun2(a,b):
        sum = a + b
        SUB = a - b
        return sum, SUB
        ret = fun2(10, 5)
        print(ret) # 输出 (15, 5), 返回一个元组
        ret1, ret2 = fun2(11, 22) # 使用元组解包来获取返回的两个值
        print(ret1, ret2) # 输出 33 11

```

(15, 5)

33 -11

嵌套函数与闭包

嵌套函数：函数里再定义函数。内部函数只能在外部函数内部调用，外面找不到。

作用域方向：内部可以访问外部的变量，外部不能访问内部的变量——单向可见。

闭包三个条件：①有嵌套函数 ②内部函数引用外部函数的变量 ③外部函数返回内部函数名（不加括号）。

闭包 vs 普通函数：普通函数执行完变量就释放了；闭包能保持外部变量状态，只要内部函数对象没被回收，外部变量就一直被引用。

入门阶段定位：能看懂闭包结构就行，不需要自己写。"函数可以携带状态"这个认知要留住——装饰器、回调都是它的一脉相承。

```
In [41]: #嵌套函数
def outer_func(x):
    def inner_func(y):
        return x + y
    return inner_func
add_eight = outer_func(8) # 创建一个函数，固定 x 为 8
result = add_eight(10) # 调用内层函数，传入 y 为 10
print(result) # 输出 18，因为 8 + 10 = 18
```

18

```
In [42]: #闭包示例
def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier
times3 = make_multiplier(3) # 创建一个函数，固定 n 为 3
result = times3(10) # 调用内层函数，传入 x 为 10
print(result) # 输出 30，因为 10 * 3 = 30
```

30

冒泡排序：第一个算法 核心规则：相邻元素两两比较，前面比后面大就交换，每轮把最大值"冒泡"到最右。手动推导（5个元素）：

- 第1轮：比较4次，确定最大值在最右
- 第2轮：比较3次，确定第二大值
- 第3轮：比较2次
- 第4轮：比较1次，排序完成 关键理解：外层控制行（轮次），内层控制列（每轮比较）。内层循环次数 = 列表长度 - 1 - 外层计数器。理解不了减j 也没关系，不减只是多几次无效比较，不影响结果。选择排序（Selection Sort）是一种简单直观的排序算法。它的核心思想非常符合人类的直觉：每一轮都从剩下的未排序元素中"挑选"出最小（或最大）的一个，放到已排序序列的末尾。

```
In [ ]: #冒泡排序
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# 外层循环控制所有的遍历
# 内层循环控制每次遍历的
# 如果前一个元素大于后一
# 交换元素
# 返回排序后的数组
```

```

arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = bubble_sort(arr) # 调用冒泡排序函数
print(sorted_arr) # 输出 [11, 12, 22, 25,

#选择排序
def selection_sort(arr):
    n = len(arr)
    for i in range(n): # 外层循环控制所有的遍历次数
        min_idx = i # 假设当前元素是最小的
        for j in range(i+1, n): # 内层循环从当前元素的下一个
            if arr[j] < arr[min_idx]: # 如果找到更小的元素, 更新最
                min_idx = j # 交换当前元素和最小元素
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # 交换元素
    return arr # 返回排序后的数组
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = selection_sort(arr)
print(sorted_arr) # 输出 [11, 12, 22, 25, 34, 64, 90]

#快速排序
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[0]
        less_than_pivot = [x for x in arr[1:] if x <= pivot]
        greater_than_pivot = [x for x in arr[1:] if x > pivot]
        return quick_sort(less_than_pivot) + [pivot] + quick_sort(greater_than_p
arr = [10, 7, 8, 9, 1, 5]
sorted_arr = quick_sort(arr)
print(sorted_arr) # 输出 [1, 5, 7, 8, 9, 10]

```

```

[11, 12, 22, 25, 34, 64, 90]
[11, 12, 22, 25, 34, 64, 90]
[1, 5, 7, 8, 9, 10]

```

```

In [48]: #作业1定义一个函数,实现返回三个数的和
def sum_of_three(a, b, c):
    return a + b + c
result = sum_of_three(1, 2, 3)
print(result) # 输出 6

```

6

```

In [50]: #作业2定义一个函数,实现返回三个数的最大值
def max_of_three(a, b, c):
    return max(a, b, c)
result = max_of_three(1, 3, 5)
print(result) # 输出 5

```

5

```

In [51]: #作业3定义一个函数,实现返回三个数的最小值
def min_of_three(a, b, c):
    return min(a, b, c)
result = min_of_three(1, 3, 5)
print(result) # 输出 1

```

1

```

In [52]: #作业4给定列表 nums = [10, 20, 30, 50, 20], 定义一个函数找出给定元素的所有位置
def find_all_positions(nums, target):
    positions = [] # 用于存储目标元素的所有位置

```

```

    for index, num in enumerate(nums): # 使用 enumerate 获取元素及其索引
        if num == target: # 如果当前元素等于目标元素
            positions.append(index) # 将索引添加到位置列表中
    return positions # 返回所有位置的列表
nums = [10, 20, 30, 50, 20]
target = 20
positions = find_all_positions(nums, target) # 调用函数并获取结果
print(positions) # 输出 [1, 4], 因为 20 出现在索引 1 和 4 的位置

```

[1, 4]

In [53]: #作业5使用for循环实现99乘法表

```

for i in range(1, 10): # 外层循环控制行数, 从 1 到 9
    for j in range(1, i + 1): # 内层循环控制列数, 从 1 到当前行数 i
        print(f"{j} x {i} = {i * j}", end="\t") # 打印乘法表项, 使用制表符分隔
    print() # 每完成一行后换行

```

```

1 x 1 = 1
1 x 2 = 2      2 x 2 = 4
1 x 3 = 3      2 x 3 = 6      3 x 3 = 9
1 x 4 = 4      2 x 4 = 8      3 x 4 = 12      4 x 4 = 16
1 x 5 = 5      2 x 5 = 10     3 x 5 = 15     4 x 5 = 20     5 x 5 = 25
1 x 6 = 6      2 x 6 = 12     3 x 6 = 18     4 x 6 = 24     5 x 6 = 30     6
x 6 = 36
1 x 7 = 7      2 x 7 = 14     3 x 7 = 21     4 x 7 = 28     5 x 7 = 35     6
x 7 = 42      7 x 7 = 49
1 x 8 = 8      2 x 8 = 16     3 x 8 = 24     4 x 8 = 32     5 x 8 = 40     6
x 8 = 48      7 x 8 = 56     8 x 8 = 64
1 x 9 = 9      2 x 9 = 18     3 x 9 = 27     4 x 9 = 36     5 x 9 = 45     6
x 9 = 54      7 x 9 = 63     8 x 9 = 72     9 x 9 = 81

```

In [55]: #作业6定义一个函数,实现返回一个数的阶乘

```

def factorial(n):
    if n == 0 or n == 1: # 阶乘的定义: 0! 和 1! 都等于 1
        return 1
    else:
        return n * factorial(n - 1) # 递归调用函数计算阶乘
result = factorial(8) # 调用函数并获取结果
print(result) # 输出 40320, 因为 8! = 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 40320

```

40320

In []: #作业7 定义一个函数, 求出 $1 + 2! + 3! + 4! + \dots + 20!$ 的结果

```

def sum_of_factorials(n):
    total = 0
    factorial_cache = {} # 用于缓存已经计算过的阶乘结果
    for i in range(1, n + 1):
        if i in factorial_cache: # 如果阶乘结果已经缓存, 直接使用
            total += factorial_cache[i]
        else:
            factorial_result = factorial(i) # 计算阶乘
            factorial_cache[i] = factorial_result # 将结果缓存起来
            total += factorial_result # 累加到总和中
    return total
result = sum_of_factorials(20) # 调用函数并获取结果
print(result) # 输出 2561327494111820313

```

2561327494111820313

In []: #作业8求前 K 个高频元素设计对应的函数

```

from collections import Counter

```

```
def top_k_frequent(nums, k):  
    count = Counter(nums) # 统计每个元素的频率  
    most_common = count.most_common(k) # 获取前 K 个最常见的元素及其频率  
    return [item[0] for item in most_common] # 返回前 K 个高频元素的列表  
nums7 = [2, 3, 1, 2, 3, 3, 3, 4, 4, 4]  
k = 2  
result = top_k_frequent(nums7, k) # 调用函数并获取结果  
print(result) # 输出 [3, 4], 因为 3 出现了4次, 4 出现了3次, 是前 2 个高频元素
```

[3, 4]